

Full Stack Development with MERN

DOCSPOT — Medical Appointment Management System

1. Introduction

1.1 Project Title

Project Title: DocSpot – MERN-Based Medical Scheduling Platform

1.2 Member

Member: Amesh J

Role: Full Stack Developer

Responsibilities

- Developed the entire user-facing interface using React for patients, doctors, and administrators.
- Created UI forms, dashboards, tables, and reusable components to offer a smooth interaction flow.
- Linked the frontend with backend services through Axios-based API calls.
- Implemented secure login and permission-based features using JWT authentication.
- Designed MongoDB schema structures for patient, doctor, and appointment-related information.
- Developed REST API endpoints using Node.js and Express to support core operations.
- Enabled upload functionality for medical reports attached to appointment requests.
- Handled UI-level state logic, data validation, and error display across the system.
- Executed comprehensive manual testing to ensure stable and correct feature behavior.
- Prepared complete project documentation and maintained the overall codebase organization.

2. Project Overview

2.1 Purpose

- DocSpot is a MERN-driven web platform designed for streamlined doctor appointment handling. The system allows:
- Patients to create accounts, sign in, explore verified doctors, attach medical files, and request consultations.
- Doctors to access pending appointment requests, approve or decline them, and review attached reports.
- Admins to manage system users, verify doctor profiles, and oversee all appointment activities.
- The system's goal is to minimize manual appointment coordination, reduce call-based scheduling, and enhance clarity in the booking cycle — centralizing patient–doctor interaction while making the entire appointment process efficient.

2.2 Project Goals

- Provide a user-friendly dashboard allowing patients to organize and track appointments.
- Offer an organized doctor interface to manage patient requests and view submitted documents.
- Create an admin environment for managing user permissions, approving doctor accounts, and reviewing system appointments.
- Guarantee secure user authentication with JSON Web Tokens (JWT).
- Use MongoDB to store and manage information in a scalable, flexible format.

2.3 Key Features

For Patients (role: user)

- Registration & Login: register using personal details such as name, email, password, phone, and role; protected login using JWT.
- Doctor Browsing: access a list of verified doctors along with specialization, background, fee, availability, and location details.
- Requesting Appointments: choose a doctor, pick a suitable date/time, and upload supporting documents (images).
- Track Appointment Status: view the full list of appointments with doctor name, date, and progression (pending, approved, etc.).

- Notifications: receive updates whenever appointment statuses get modified.

For Doctors (isdoctor = true)

- Doctor Application: submit a profile including specialization, experience, consultation fee, and available timings.
- Manage Requests: review appointment entries with patient information and supporting documents, then approve them.
- Document Review: download medical documents attached by patients for analysis.

For Admin (role: admin)

- Admin Dashboard: system-wide centralized appointment view.
 - User Controls: manage all accounts and their access permissions.
 - Doctor Approvals: review submitted doctor profiles and either approve or deny them.
 - Admin Notifications: access notifications for major system events such as new doctor applications.
-

3. Architecture

The system is constructed using the MERN stack:

Frontend: React.js

Backend: Node.js + Express.js

Database: MongoDB

Interface: REST APIs exchanging JSON data

3.1 Frontend Architecture (React)

The client interface is a single-page React application.

Routing includes:

/ – Application landing page

/login – Sign-in interface

/register – Registration screen

/userhome – Dashboard for users (patients/doctors)

/adminhome – Dashboard for administrators

Folder setup:

- components/admin – contains admin-related modules
- components/common – includes shared modules (Home, Login, Register, Notification)
- components/user – contains user and doctor dashboard screens

State handling:

Utilizes React useState and useEffect for handling UI data and side effects.

Saves logged-in details in localStorage as userData.

Stores the JWT token in localStorage and attaches it to all protected API calls.

UI Libraries:

React-Bootstrap (cards, navbars, modals, tables)

MDB React UI Kit (authentication layout components)

Material UI Icons (used for dashboard sidebar navigation)

3.2 Backend Architecture (Node.js + Express)

The backend provides RESTful services mapped under /api/user, /api/admin, and /api/doctor.

Main Layers:

- **Server Entry (server.js / index.js):** initializes Express, connects to MongoDB, loads routes, and enables middleware.
- **Routes:**
 - userRoutes.js: register, login, fetch user info, doctor application, list doctors, create appointment, notifications
 - adminRoutes.js: retrieve user list, doctor list, approve/reject, and view all appointments

- doctorRoutes.js: doctor-specific appointment listing, update status, download documents
 - **Controllers:** implement business logic and interact with MongoDB models.
 - **Middleware:** JWT verification middleware attaches decoded information to req.user.
 - **Models:** Mongoose schemas for User, Doctor, Appointment, Notification, and file data.
-

3.3 Database Architecture (MongoDB)

Collections store flexible JSON documents.

Key collections:

Users:

_id, fullName, email, password, phone, type, isdoctor, notification, seennotification

Doctors:

_id, userId, fullName, email, phone, address, specialization, experience, fees, timings, status

Appointments:

_id, userId, doctorId, userInfo, doctorInfo, date, status, document

Notifications:

type, message, onClickPath stored inside notification arrays

Relationships:

A user may hold a linked doctor profile (if isdoctor is true).

A doctor may manage several appointments.

A patient can have multiple booking entries.

4. Setup Instructions

4.1 Prerequisites

Node.js (recommended v14 or above)

npm or Yarn package manager

MongoDB Community Edition or MongoDB Atlas cloud instance

Git installed (optional but useful for cloning the repository)

4.2 Installation & Environment Setup

The project contains two main folders: client (React frontend) and server (Express backend).

Step 1: Clone

```
git clone https://github.com/your-username/caresync-portal.git
```

```
cd caresync-portal
```

Step 2: Install frontend dependencies

```
cd client
```

```
npm install
```

Step 3: Install backend dependencies

```
cd ../server
```

```
npm install
```

Step 4: Configure .env within the server

```
PORT=8001
```

```
MONGO_URL=mongodb://localhost:27017/caresync
```

```
JWT_SECRET=your_jwt_secret_key_here
```

```
CLIENT_URL=http://localhost:3000
```

PORT: backend port where the API server runs

MONGO_URL: connection URI for MongoDB

JWT_SECRET: used to generate and verify JWT tokens

5. Folder Structure

5.1 Client (React)

client/

src/

App.jsx

App.css

index.jsx

components/

admin/

AdminHome.jsx

AdminUsers.jsx

AdminDoctors.jsx

AdminAppointments.jsx

common/

Home.jsx

Login.jsx

Register.jsx

Notification.jsx

user/

UserHome.jsx

ApplyDoctor.jsx

DoctorList.jsx

UserAppointments.jsx

images/

Notes

App.jsx manages routing and renders different pages.
App.css includes global styles and dashboard UI rules.
components/admin contains admin dashboards and control screens.
components/common holds shared pages such as login, register, and notifications.
components/user contains pages for patients and doctors, including applications and appointments.

5.2 Server (Node.js)

server/

server.js

package.json

.env

config/

db.js

models/

User.js

Doctor.js

Appointment.js

routes/

userRoutes.js

adminRoutes.js

doctorRoutes.js

controllers/

UserController.js

AdminController.js

doctorController.js

middleware/

authMiddleware.js

uploads/

documents/

Notes

server.js initializes Express, binds middleware, and configures MongoDB.

config/db.js manages Mongoose database connection logic.

models/ includes schemas for all database entities.

routes/ map URL endpoints to corresponding controllers.

middleware/authMiddleware.js ensures only valid JWTs can access protected APIs.

uploads/documents contains saved medical files uploaded by users.

6. Running the Application

6.1 Start Backend

Open the server directory:

```
cd server
```

```
npm start
```

Backend normally runs at <http://localhost:8001>.

6.2 Start Frontend

Open the React frontend:

```
cd client
```

```
npm start
```

Frontend loads at <http://localhost:3000>.

Tip: Always launch the backend before opening the frontend so API requests function properly.

7. API Documentation

7.1 User APIs (/api/user)

Register

- **POST /api/user/register**
- Payload: fullName, email, password, phone, type
- Response: confirmation message indicating successful registration

Login

- **POST /api/user/login**
- Payload: email, password
- Response: success flag, JWT token, userData

Get User Data

- **POST /api/user/getuserdata**
- Header: Authorization: Bearer <token>
- Function: returns authenticated user information

Apply for Doctor Profile

- **POST /api/user/registerdoc**
- Header: Authorization
- Body: { doctor: {...}, userId }
- Purpose: submit doctor application details

Get Approved Doctors

- **GET /api/user/getalldoctorsu**
- Header: Authorization
- Function: fetch list of all approved doctors

Create Appointment

- **POST /api/user/getappointment**
- Headers: Authorization, Content-Type: multipart/form-data

- Body: image, date, userId, doctorId, userInfo, doctorInfo
- Function: create new appointment request

Get User Appointments

- **GET /api/user/getuserappointments?userId=<id>**
- Header: Authorization
- Description: retrieves appointments tied to the specific user

Get All Notifications

- **POST /api/user/getallnotification**
- Header: Authorization
- Body: { userId }
- Description: moves unread alerts to the seen section

Delete All Notifications

- **POST /api/user/deleteallnotification**
- Header: Authorization
- Body: { userId }
- Function: clears all notifications for the selected user

7.2 Admin APIs (/api/admin)

Get All Users

- **GET /api/admin/getallusers**
- Header: Authorization

Get All Doctors

- **GET /api/admin/getalldoctors**
- Header: Authorization

Approve Doctor

- **POST /api/admin/getapprove**

- Header: Authorization
- Body: { doctorId, status: 'approved', userid }

Reject Doctor

- **POST /api/admin/getreject**
- Header: Authorization
- Body: { doctorId, status: 'rejected', userid }

Get All Appointments (Admin)

- **GET /api/admin/getallAppointmentsAdmin**
- Header: Authorization

7.3 Doctor APIs (/api/doctor)

Get Doctor Appointments

- **GET /api/doctor/getdoctorappointments?userId=<id>`**
- Header: Authorization

Handle Appointment Status

- **POST /api/doctor/handlestatus**
- Header: Authorization
- Body: { userid, appointmentId, status }

Download Document

- **GET /api/doctor/getdocumentdownload?appointId=<id>`**
- Header: Authorization

8. Authentication

8.1 Method

The application uses **JWT (JSON Web Token)** authentication to manage secure access.

Login Process

1. Client submits email and password to `/api/user/login`.
 2. Backend checks credentials; if valid, it generates a signed JWT.
 3. Token is saved in `localStorage` by the frontend.
 4. Protected routes include `Authorization: Bearer <token>` in headers.
 5. Backend middleware verifies token and attaches decoded data to `req.user`.
-

8.2 Roles & Authorization

- **Admin (type = 'admin'):** full access to admin-specific features and dashboard.
- **User (type = 'user'):** standard patient functionality.
- **Doctor (isdoctor = true):** user with verified doctor profile; can view and respond to appointment requests.

Backend access rules ensure only properly authenticated and authorized users reach protected endpoints.

9. User Interface

9.1 Overview

The interface aims to be user-friendly, responsive, and clear. It relies on:

- React-Bootstrap for layout structure
 - Ant Design for notifications, tabs, and badges
 - Custom CSS for dashboard styling and layout behavior
-

9.2 Pages Overview

Landing Page (Home.jsx)

- Hero section introducing DocSpot and its purpose

- Prominent buttons for “Book an appointment” and “Join as patient or doctor”
 - Short “About” block describing overall system benefits
-

Login Page

- Header with navigation links
 - Illustrated layout with form on one side
 - Fields include email, password, login button, and link to registration screen
-

Register Page

- Form collects name, email, password, phone, and role selection
 - Includes submit button labeled “Create account”
 - Message linking back to login page
-

User Dashboard (UserHome.jsx)

- Sidebar showing navigation links such as “My Appointments” and “Apply as Doctor”
 - Header showing user’s name and notifications count
 - Main content switches between appointment list, doctor application, notifications, or doctor list
-

Doctor List View

- Doctors displayed as cards with full details
 - “Request Appointment” opens a modal to submit booking details
-

Appointment Request Modal

- Shows selected doctor details
 - Contains date and time picker plus file upload for report submission
 - Buttons for submission or cancellation
-

Doctor Dashboard (UserAppointments as Doctor)

- Table listing patient appointment requests
 - Shows patient name, contact, appointment time, attached documents, and current status
 - Includes options to approve and download reports
-

Admin Dashboard (AdminHome.jsx)

- Sidebar with options for appointments, users, and doctor approvals
 - Header with notification icon
 - Displays system-wide tables for appointments, user data, and doctor verification
-

Admin — Doctor Management

- Table listing doctor accounts with names, contact info, and status
 - Admin can approve or reject submitted doctor profiles
 - On action, table refreshes and shows updated status
-

10. Testing

10.1 Testing Strategy

Testing efforts focused primarily on manual validation with selective automated scenarios.

Types of Testing

Unit Testing

- Concentrated on backend logic inside controllers and route functions.
- Validated registration, login operations, doctor approval behavior, and appointment creation.

Integration Testing

- Confirmed end-to-end flow between frontend requests and backend responses.
- Example: creating an appointment from the UI correctly generated a database entry and updated doctor/admin views.

Functional Testing

- Verified each major feature including:
 - User registration
 - User login
 - Doctor application workflow
 - Appointment requests
 - Notifications
 - Admin approval system

UI/UX Testing

- Checked responsiveness across breakpoints.
- Ensured correct display of validation messages and state-based errors.

10.2 Tools

- **Postman** — used to validate REST endpoints
- **MongoDB Compass** — inspected stored documents

- **Browser Developer Tools** — debugged frontend behavior and network activity

11. Demo

A demonstration video is provided and can be included when sharing the documentation.

File: [*DocSpot.mp4*](#)

12. Known Issues

A few existing limitations have been identified:

Validation

- Standard checks are implemented, but stronger constraints (email format, password strength) could enhance reliability.

File Uploads

- Currently optimized for image types; PDF upload may require additional backend adjustments.

Mobile Optimization

- Interfaces are responsive, though certain complex tables may need extra tuning for very small mobile screens.

Rate Limiting & Security

- No specialized rate-limit mechanism is active, so brute-force protection can be improved.

Error Messages

- Some fallback messages such as “Something went wrong” could be replaced with more descriptive, user-friendly feedback.
-

13. Future Enhancements

Potential improvements for later releases include:

1. Advanced Analytics for Admin

- Dashboard metrics showing active doctors, appointments, and usage trends.

2. Email/SMS Notifications

- Automatic reminders for upcoming appointments.

3. Rescheduling & Cancellation

- Allow patients to change or cancel appointments with instant doctor notifications.

4. Secure Chat System

- Built-in encrypted messaging between patient and doctor for pre-consultation discussions.

5. Payment Gateway Integration

- Online consultation fee processing through platforms like Razorpay or Stripe.

6. Multi-Clinic Support

- Expand system to handle multiple clinics with individual schedules.

7. Additional User Roles

- Introduce roles such as receptionist or clinic manager with limited access permissions.

8. Server Logging & Monitoring

- Add auditing tools and monitoring dashboards for improved maintenance and traceability.